

## DBMS – WEEK 4

### TATA CLiQ E-COMMERCE DATABASE MANAGEMENT SYSTEM

#### Orders and Order\_Details Management

##### 1. Objective

1. To design the Orders and Order\_Details tables.
2. To manage customer product orders.
3. To store order date, quantity and total amount.
4. To perform order insertion and modification operations.
5. To generate customer order history and customer-wise order summary reports.

##### 2.Database Design

###### 2.1 Orders Table

| Attribute    | Key         | Type         |
|--------------|-------------|--------------|
| Order_ID     | Primary Key | NUMBER(5)    |
| Customer_ID  | Foreign Key | NUMBER(5)    |
| Order_Date   | —           | DATE         |
| Total_Amount | —           | NUMBER(10,2) |

###### 2.2 Orders Details Table

| Attribute       | Key         | Type         |
|-----------------|-------------|--------------|
| Order_Detail_ID | Primary Key | NUMBER(5)    |
| Order_ID        | Foreign Key | NUMBER(5)    |
| Product_ID      | Foreign Key | NUMBER(5)    |
| Quantity        | —           | NUMBER(5)    |
| Total_Amount    | —           | NUMBER(10,2) |

### 3. Create Orders Table

```
SQL> CREATE TABLE Orders (  
2     Order_ID NUMBER(5),  
3     Customer_ID NUMBER(5),  
4     Order_Date DATE,  
5     Total_Amount NUMBER(10,2)  
6 );
```

Table created.

### 4. Create Order Details Table

```
SQL>  
SQL> CREATE TABLE Order_Details (  
2     Order_Detail_ID NUMBER(5),  
3     Order_ID NUMBER(5),  
4     Product_ID NUMBER(5),  
5     Quantity NUMBER(5),  
6     Total_Amount NUMBER(10,2)  
7 );
```

Table created.

### 5. Order Data Insertion

```
SQL> INSERT INTO Orders VALUES  
2 (1001, 101, TO_DATE('20-09-2026', 'DD-MM-YYYY'), 2500);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Orders VALUES  
2 (1002, 102, TO_DATE('21-09-2026', 'DD-MM-YYYY'), 1800);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Orders VALUES  
2 (1003, 103, TO_DATE('22-09-2026', 'DD-MM-YYYY'), 3200);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Orders VALUES  
2 (1004, 101, TO_DATE('23-09-2026', 'DD-MM-YYYY'), 1500);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Orders VALUES  
2 (1005, 104, TO_DATE('24-09-2026', 'DD-MM-YYYY'), 4200);
```

## 6.Order Details Data Insertion

```
SQL> INSERT INTO Order_Details VALUES  
2 (1, 1001, 201, 2, 2000);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Order_Details VALUES  
2 (2, 1001, 202, 1, 500);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Order_Details VALUES  
2 (3, 1002, 203, 2, 1800);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Order_Details VALUES  
2 (4, 1003, 204, 2, 3200);
```

1 row created.

```
SQL>  
SQL> INSERT INTO Order_Details VALUES  
2 (5, 1004, 205, 1, 1500);
```

```
SQL> SELECT * FROM Order_Details;
```

| ORDER_DETAIL_ID | ORDER_ID | PRODUCT_ID | QUANTITY | TOTAL_AMOUNT |
|-----------------|----------|------------|----------|--------------|
| 1               | 1001     | 201        | 2        | 2000         |
| 2               | 1001     | 202        | 1        | 500          |
| 3               | 1002     | 203        | 2        | 1800         |
| 4               | 1003     | 204        | 2        | 3200         |
| 5               | 1004     | 205        | 1        | 1500         |
| 6               | 1005     | 206        | 2        | 4200         |

6 rows selected.

## 7. Order Modification Operations

```
SQL>
SQL> SELECT * FROM Orders
  2  WHERE Order_ID = 1002;

ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
-----
1002      102 21-SEP-26      3000

SQL> UPDATE Orders
  2  SET Order_Date = TO_DATE('25-09-2026', 'DD-MM-YYYY')
  3  WHERE Order_ID = 1002;

1 row updated.

SQL>
SQL> COMMIT;

Commit complete.

SQL>
SQL> SELECT * FROM Orders
  2  WHERE Order_ID = 1002;

ORDER_ID CUSTOMER_ID ORDER_DAT TOTAL_AMOUNT
-----
1002      102 25-SEP-26      3000
```

## 8. Order\_Details Modification

```
SQL> UPDATE Order_Details
  2  SET Quantity = 3
  3  WHERE Order_Detail_ID = 1;

1 row updated.

SQL> COMMIT;

Commit complete.

SQL> SELECT * FROM Order_Details
  2  WHERE Order_Detail_ID IN (1, 2);

ORDER_DETAIL_ID  ORDER_ID  PRODUCT_ID  QUANTITY  TOTAL_AMOUNT
-----
1                1001      201        3         2000
2                1001      202        1          500

SQL> UPDATE Order_Details
  2  SET Total_Amount = 700
  3  WHERE Order_Detail_ID = 2;

1 row updated.

SQL> SELECT * FROM Order_Details
  2  WHERE Order_Detail_ID IN (1, 2);

ORDER_DETAIL_ID  ORDER_ID  PRODUCT_ID  QUANTITY  TOTAL_AMOUNT
```

## 9. Customer Order History Report

```
SQL> SELECT
  2     O.Order_ID,
  3     O.Customer_ID,
  4     O.Order_Date,
  5     OD.Product_ID,
  6     OD.Quantity,
  7     OD.Total_Amount
  8 FROM Orders O
  9 JOIN Order_Details OD
10 ON O.Order_ID = OD.Order_ID;
```

| ORDER_ID | CUSTOMER_ID | ORDER_DAT | PRODUCT_ID | QUANTITY | TOTAL_AMOUNT |
|----------|-------------|-----------|------------|----------|--------------|
| 1001     | 101         | 20-SEP-26 | 201        | 3        | 2000         |
| 1001     | 101         | 20-SEP-26 | 202        | 1        | 700          |
| 1002     | 102         | 25-SEP-26 | 203        | 2        | 1800         |
| 1003     | 103         | 22-SEP-26 | 204        | 2        | 3200         |
| 1004     | 101         | 23-SEP-26 | 205        | 1        | 1500         |
| 1005     | 104         | 24-SEP-26 | 206        | 2        | 4200         |

6 rows selected.

## 10. Customer-wise Order Summary

```
SQL> SELECT
  2     Customer_ID,
  3     COUNT(Order_ID) AS Total_Orders,
  4     SUM(Total_Amount) AS Total_Amount
  5 FROM Orders
  6 GROUP BY Customer_ID
  7 ORDER BY Customer_ID;
```

| CUSTOMER_ID | TOTAL_ORDERS | TOTAL_AMOUNT |
|-------------|--------------|--------------|
| 101         | 2            | 4200         |
| 102         | 1            | 3000         |
| 103         | 1            | 3200         |
| 104         | 1            | 4200         |

SQL>

**Conclusion:**

The Orders and Order\_Details tables were successfully designed and implemented for the Tata CLiQ E-Commerce Database Management System. Order records and order detail records were inserted and modified successfully. Customer order history and customer-wise order summary reports were generated using SQL queries. A join operation was also performed between the Orders and Order\_Details tables.